

TinyTorch Reflections — Answer Format

ML Systems Reflection Questions with structured answers

Exactly. That is a much better learning format for your TinyTorch reflections. It forces you to understand the mechanism instead of memorizing the answer.

Use this structure for every question:

Concept → Why → Example → Rule

1. Memory Layout & Cache Performance

How does row-major vs. column-major storage affect cache performance in tensor operations?

Consider:

- What happens when you access matrix elements sequentially vs. with large strides?
- Why did our analysis show column-wise access being slower than row-wise?
- How would this affect the design of an image processing pipeline's memory layout?

Concept	Row-major vs. column-major storage
Why	Contiguous memory access uses the CPU cache more efficiently than strided access.
Example	In row-major storage, row-wise access is contiguous and usually faster than column-wise access.
Rule	Access data in the order it is stored to maximize cache locality.

Key Insight: Libraries choose specific memory formats because accessing certain dimensions sequentially has better cache locality — a principle applied throughout later modules.

2. Batch Processing & Scaling

If you double the number of samples in a batch, what happens to memory usage? What about computation time?

Consider:

- An affine transformation with input (batch, features): $y = xW + b$
- Memory for: input tensor, weight matrix, output tensor
- If (32, 784) @ (784, 256) takes 10ms, how long does (64, 784) @ (784, 256) take?

Concept	Batch size scaling
Why	More samples require more memory and more computation.
Example	If batch 32 takes 10 ms, batch 64 takes approximately 20 ms.

Rule Memory and computation scale approximately linearly with batch size.

Key Insight: Input/output memory scales linearly with batch size, but weight memory stays constant. Computation time also scales linearly for matrix multiplication.

3. Data Type Precision & Memory

What's the memory difference between float64 and float32 for a (1000, 1000) tensor?

Calculate:

- float64: 8 bytes per element, float32: 4 bytes per element
- Total elements: 1,000,000
- Memory: float64 = 8MB, float32 = 4MB (2x difference)

Concept Numerical precision affects memory

Why Larger data types require more bytes per element.

Example A (1000,1000) tensor uses 8 MB with float64 and 4 MB with float32.

Rule Lower precision reduces memory usage but may reduce numerical precision.

Key Insight: Production systems often use float16 or bfloat16 for 4x memory savings over float32, trading precision for capacity. GPU memory limits (8–16GB) make this critical.

4. Production-Scale Memory

A large-scale model has 175 billion parameters. How much RAM is needed just to store the weights?

Calculate:

- Parameters: 175×10^9
- Bytes per float32: 4
- Weight memory: 700 GB

Concept Parameter count determines weight memory

Why Every parameter must be stored in memory.

Example 175B parameters $\times$ 4 bytes (float32) = 700 GB.

Rule Large models require substantial memory even before gradients and optimizer states.

Key Insight: This is why large-scale systems require significant hardware resources. "Additional training state" (gradients, optimizer states) is explored in later modules.

5. Hardware Awareness

Why do parallel processors strongly prefer operations on large tensors over many small ones?

Compare:

- Scenario A: 1000 separate (10, 10) matrix multiplications
- Scenario B: 1 batched (1000, 10, 10) matrix multiplication

Concept	Batching improves parallelism
Why	Many small operations incur repeated launch overhead, while batching amortizes that cost.
Example	One (1000,10,10) batched operation is generally more efficient than 1000 separate (10,10) operations.
Rule	Batch operations to reduce overhead and maximize hardware utilization.

Key Insight: Computation launch overhead (~5–10 microseconds per launch) dominates for small operations. Batching amortizes this overhead and maximizes parallelism across processing units.

This is the format I'd recommend you continue using throughout TinyTorch: short, technical, and focused on understanding the underlying system behavior.